

CPU

Datapath

- ALU
- Registers:
 - Low Registers R0 – R7
 - High Registers R8 – R12
 - Stack Pointer (R13)
 - Link Register (R14) = Return from procedures
 - Program Counter (R15) = Address of next instruction
- Flags
 - N: Negative, Z: Zero
 - C: ADD→Carry, SUBS→No Borrow (Unsigned)
 - V: Overflow/Underflow (= Carry OR Carry to highest Bit) (Signed)
- Control Unit: Contains Instruction Register with Opcode
- Bus Interface (contains registers to store addresses)

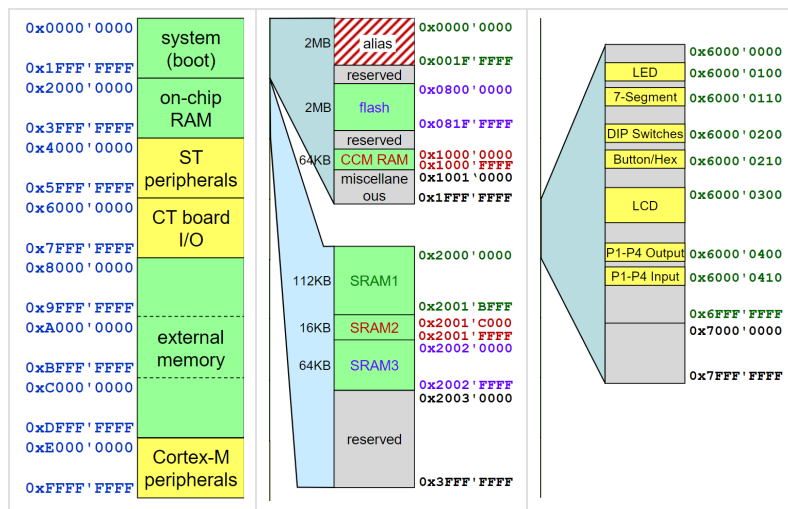
Control Unit: State machine of reset, fetch, execute

Program Execution

1. "Reset": Read 0x0000'0004 into PC
2. Send PC address over bus interface
3. Get opcode over bus interface into IR
4. Increment PC
5. Execute

Main memory

volatile	non-volatile
SRAM – Static RAM DRAM – Dynamic RAM	ROM factory programmed flash in system programmable
DATA, STACK, CODE	CODE



System-Bus

- address lines
- control signals
- data lines

C to executable

1. Preprocessor: Text processing
2. Compiler: CPU-independent C-code into CPU-specific assembly code
3. Assembler: Translate to machine instructions (binary File)
4. Linker: Merge object files, Resolve dependencies (.exe)

Object File Sections

Code

```
AREA MyCode, CODE, READONLY
literal DCD 0x7261504f
```

Data

- Global & static variables → AREA GlobalVars, DATA, READWRITE
- Constant variables → AREA MyConsts, DATA, READONLY

```
AREA MyData, DATA, READWRITE
var1 DCB 0x1A
var2 DCB 0x2B, 0x3C, 0x4D, 0x5E
var3 DCW 0x6F70, 0x8192
var4 DCD 0xA3B4C5D6
data1 SPACE 256
```

Stack

```
AREA STACK, NOINIT, READWRITE
```

Loading Data

C-Type – unsigned integers	Size	Term	inttypes.h / stdint.h
unsigned char	8 Bit	Byte	uint8_t
unsigned short	16 Bit	Half-word	uint16_t
unsigned int	32 Bit	Word	uint32_t
unsigned long	32 Bit	Word	uint32_t
unsigned long long	64 Bit	Double-word	uint64_t

C-Type – signed integers	Size	Term	inttypes.h / stdint.h
signed char	8 Bit	Byte	int8_t
short	16 Bit	Half-word	int16_t
int	32 Bit	Word	int32_t
long	32 Bit	Word	int32_t
long long	64 Bit	Double-word	int64_t

0x5E6F in little endian ⇒ 6F 5E (LSB)

Sign extension signed ⇒ extend with MSB

Unsigned Tructation = Modulo 2^x

\$ = EXOR, # = OR

Clear Bits: ??? Set Bits: OR Invert Bits: EXOR

```
MY_CONST1 EQU 0xFF
MY_CONST2 EQU 0xFFFFFFFF

MOVS R1,#0x01

MOVS R1,#MY_CONST1

LDR R1,=0x12345678

LDR R1,=MY_CONST2

LDR R1,mylita ; load 0xFFFFFFFF

LDR R1,=mylita ; load address of mylita (will be stored in some space)

mylita DCD 0xFFFFFFFF

byte_array DCB 0xAA,0xBB,0xCC,0xDD ; store byte array
            DCB 0xEE,0xFF

half_word DCW 0x1234 ; store half word
byte DCB 0x12 ; store byte
```

Array access

```
DATA
my_array DCD 0x11223344 ; store word array
        DCD 0x55667788
adr_w    DCD my_array      ; store address of byte array

CODE
LDR R1,my_array          ; load 0x11223344
LDR R1,adr_w             ; load address of my_array

LDR R2,=my_array         ; load address of my_array without needing adr_w
```

Multi-Word Addition ADCS

```
ADDS R1,R1,R4
ADCS R2,R2,R5
ADCS R3,R3,R6 ; R3 R2 R1 + R6 R4 R3

SUBS R1, R1, R4
SBCS R2, R2, R5
SBCS R3, R3, R6
```

1 2 4 8 16 32 64 128 256 512 1024 2048 4096 8192 16384 32768 65536 131072 262144 0 1 2 3 4

524288 1048576 2097152 4194304 8388608 16777216 33554432 67108864 134217728 268435456 536870912 1073741824 2147483648 4294967296

A B C D E F 10 11 12 13 14 15

Jump to address

```
LDR R0,=jmpaddr
BX R0 ; jump to address
```

If-then-else

```
CMP ...
BLT else
...
B end_if
else ...
end_if...
```

Do-While

```
loop ADDS R2, ...
CMP R2,.
BLT loop
```

While

```
B test
loop ADDS R2, ...
test CMP R2,.
BLT loop
```

Switch-Statement

```
NR_CASES EQU 6

case_switch CMP R1, #NR_CASES
    BHS case_default
    LSLS R1, #2
    LDR R7, =jump_table
    LDR R7, [R7, R1]
    BX R7

case_0
    ADDS R2, R2, #17
    B end_sw_case
case_1
```

```

        ADDS R2, R2, #13 ; fall through
case_3_5
        ADDS R2, R2, #37
        B end_sw_case
case_default
        MOVS R2,#0

end_sw_case ...

jump_table
        DCD case_0
        DCD case_1
        DCD case_default
        DCD case_3_5
        DCD case_default
        DCD case_3_5

```

Subroutines

```

BL label ; go to subroutine
BLX addr

BX LR ; return from subroutine

PROC
PUSH {R4,R5,LR} ; save PC at start of subroutine
; recursion possible
POP {R4,R5,PC} ; return
ENDP

```

Possible Operations:

```

ADD <Rd>,SP,#<imm>
ADD SP,SP,#<imm> ; POP = SP + 4
SUB SP,SP,#<imm> ; PUSH = SP - 4
LDR <Rt>,[SP,#<imm>]
STR <Rt>,[SP,#<imm>]

```